

Lab Description

This lab will introduce students to the principles of generating system response for single input single output systems for developing a model of the system. To generate this data, students will construct the circuit and let the output reach a steady state before implementing a step change in the input. By collecting this data students will be able to then use system identification to determine the order of the system and the relevant parameters. This hands-on experience will enhance student understanding of system response data and system identification.

Learning Outcomes

- Generate system response data to a step input
- Develop first order models using system response data

Theory

In engineering, numerous dynamic processes can be described mathematically using first-order linear differential equations, often referred to as single-capacity systems. We opted for a hydraulic analog to illustrate fundamental concepts in process dynamics. For our water tank system response experiments, we are employing a two-pump, two-tank system to simulate the behavior of a single-tank, single-valve system.

Using a linear relationship between head and flow, the discharge rate $q(t)$ then varies directly with the head (height) and inversely with the outlet resistance, $q(t) = h(t)/R$. From a material balance the differential equation and initial condition describing the tank are:

$$C \frac{dh(t)}{dt} = q_{in}(t) - q(t) \text{ at } t = 0, h = h(0) \quad [2.1]$$

in which C is the cross-sectional area of the tank and $q_{in}(t)$ is inlet volumetric flow rate. Using the flow relationship $h(t) = q(t)R$, we can express the model in terms of only h or only q . Here we choose q and obtain:

$$\frac{dq(t)}{dt} = -\frac{1}{\tau}q(t) + \frac{1}{\tau}q_{in}(t) \text{ at } t = 0, q = q(0) \quad [2.2]$$

in which $\tau = RC$ is a time constant. Defining deviation variables $\bar{q}(t) = q(t) - q(0)$ and $\bar{q}_{in}(t) = q_{in}(t) - q_{in}(0)$, and taking Laplace transforms, we can express this relationship with a transfer function, $g(s)$:

$$g(s) = \frac{\bar{q}(s)}{\bar{q}_{in}(s)} = \frac{1}{\tau s + 1} \quad [2.3]$$

Response to a Step Input

The solution to Equation 2.2 for $\bar{q}_{in}(t) = kP(t)$, where k is a constant and $P(t)$ is the unit step function, for initial conditions $q_{in}(0) = q(0)$ at $t = 0$ is:

$$\bar{q} = \bar{q}_{in}(1 - \exp(-\frac{t}{\tau})) \text{ [2.4]}$$

The solution to Equation 2.1 for the same step input $\bar{q}_{in}(t) = kP(t)$ becomes:

$$h(t) - h(0) = (h_{ss} - h(0))(1 - \exp(-\frac{t}{\tau})) \text{ [2.5]}$$

The fraction of the total change in height at time t is then

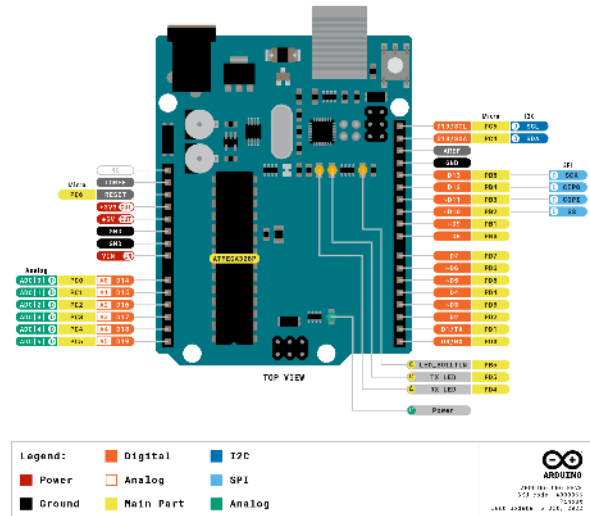
$$\frac{h(t) - h(0)}{h_{ss} - h(0)} = 1 - \exp(-\frac{t}{\tau}) \text{ [2.6]}$$

Materials Required



20kg Digital Load Cell Weight Sensor

The 20kg Digital Load Cell Weight Sensor uses a strain gauge attached to a sturdy aluminum base to measure force or weight. When force is applied, the strain gauge's resistance changes, producing a different electrical output. Note that calibration with known weights is required after installation.

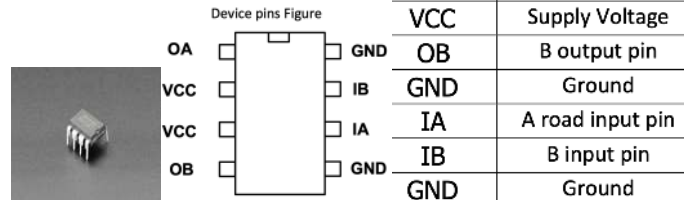


Arduino UNO

The Arduino UNO is a microcontroller board based on the ATmega328P microcontroller. It works by executing instructions stored in its memory, which are written in the Arduino programming language (based on C/C++). The board has digital and analog input/output pins that can be used to interface with various sensors and actuators. These pins can read digital or analog signals from sensors and output signals to control actuators. By writing code and uploading it to the Arduino UNO, users can create interactive electronic projects that respond to their environment.

L9110H H-Bridge Motor Driver x2

The L9110H H-Bridge Motor Driver is a chip that can control the direction and speed of a DC motor. It works by using four transistors to control the flow of current through the motor. By applying different combinations of signals to its input pins, the L9110H can make the motor turn in forward, reverse, or stop it altogether. It is commonly used in robotics and other projects requiring motor control.



NAU7802 24-Bit ADC

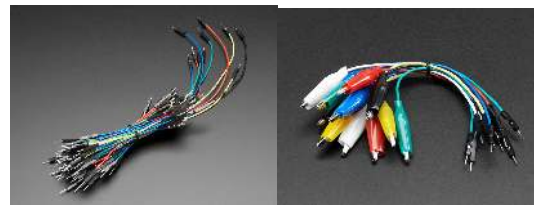
A high-resolution sensor that can measure tiny changes in voltage. It's designed to work with sensors like strain gauges or load cells, which are often used to measure stress or strain. The ADC converts these voltage changes into digital values that can be read by a microcontroller, like an Arduino.





JST SH 4-Pin to Premium Male Headers Cable

A connector cable that allows you to easily connect Qwiic-compatible devices to your projects.



Jumper Wires and Alligator Clips

Used to make temporary connections in breadboards, circuits, or between components.



Half Sized Premium Breadboard with 400 tie points

A prototyping board used to build and test electronic circuits. It consists of interconnected metal clips housed in a plastic case, providing a platform for mounting and connecting electronic components without the need for soldering.



Submersible 3V DC Water Pump
x2

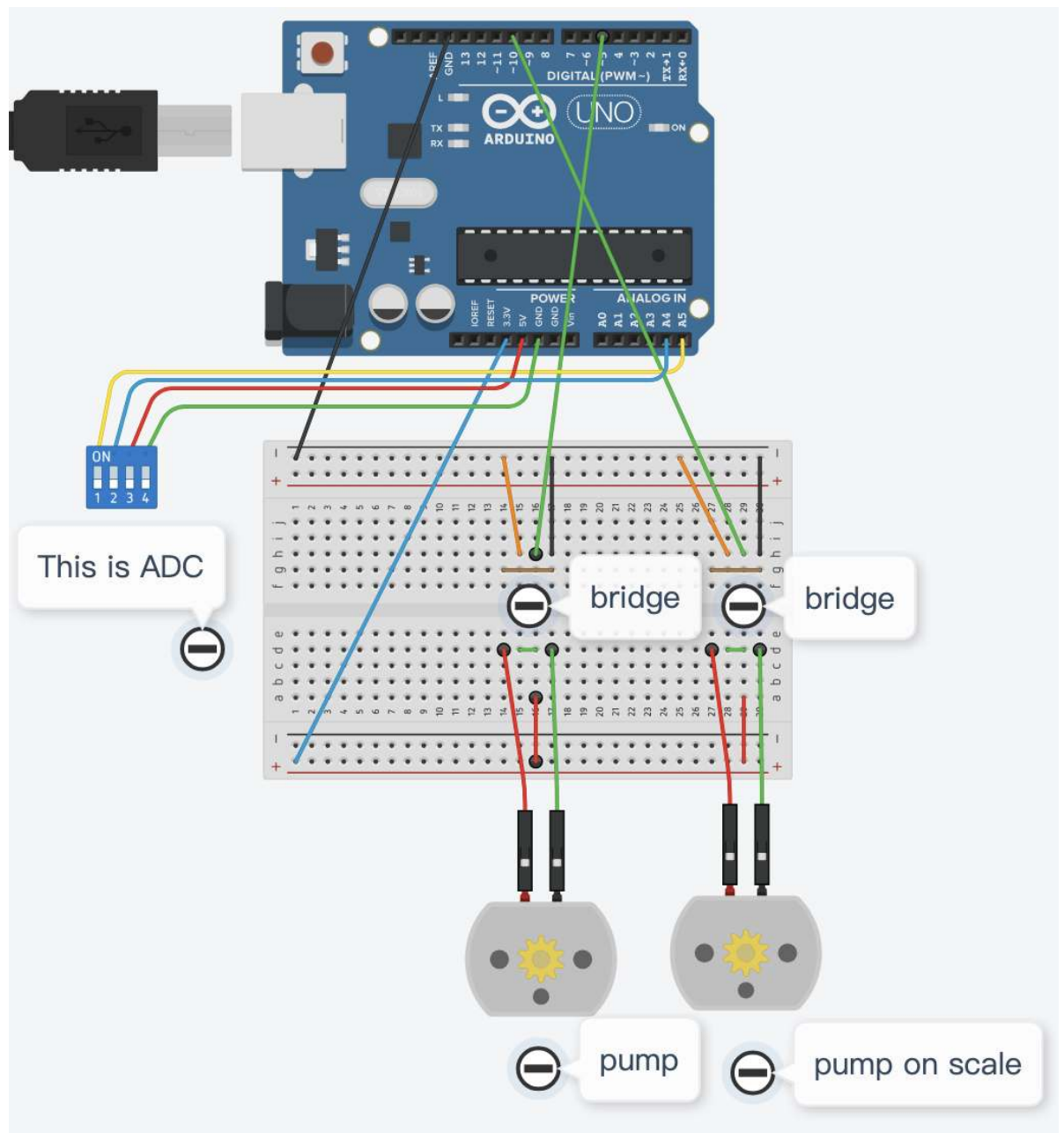
A small water pump designed to be submerged in water and operated using a 3V DC power source. Can pump water to a maximum of 1 meter high.

Other:

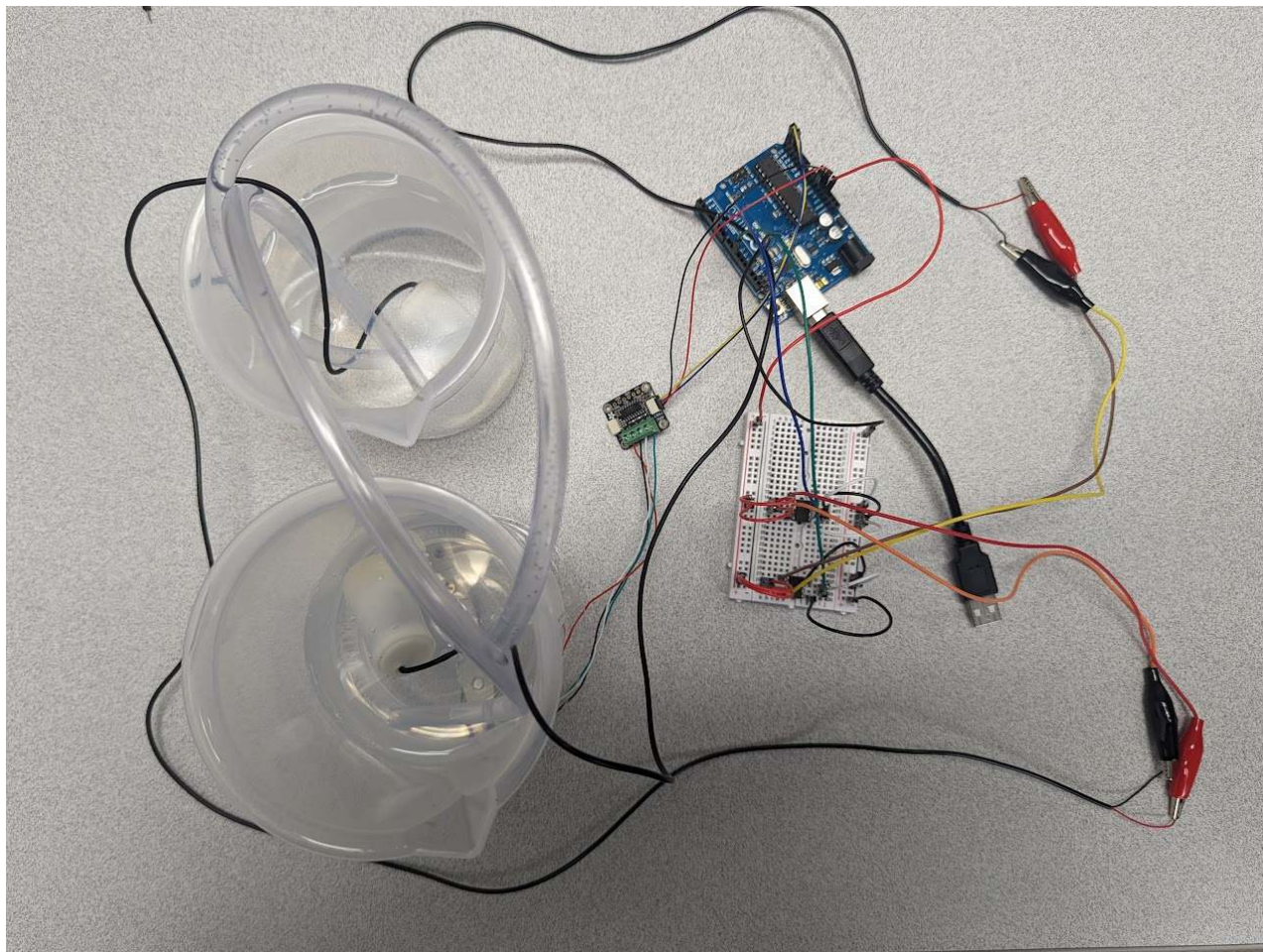
500 mL Beaker (x2), Water, and Tubing for Submersible Pumps (PVC 6mm ID)

Building the Circuit

Use the circuit diagram below to connect the pump, H-Bridge, and scale to the Arduino Uno using the breadboard. **Make sure the notch of the H-Bridge is pointing to the left for the following graph.**



The completed circuit should look something like the one below:



The structure is planned to mimic a tank with an input and valve, as shown in the diagram above.

Code Required

The Arduino Uno and Simulink communicate to one another using serial communication. The Arduino Uno does not know it is communicating with Simulink just as Simulink does not know it is communicating with the Arduino Uno. Serial communication works by just reading and writing a bit at a time. For this lab, there are two sets of Arduino sketches and Simulink simulations, the procedure for establishing communication and a description of the files is below:

To set up the serial communication, follow the steps below:

1. Download the Arduino sketch and MATLAB Simulink file.
2. Use the Arduino IDE to open the sketch.
3. Connect the Arduino Uno to the computer and select the board (make note of the COM#).
4. Upload the sketch to the Arduino Uno.
5. Open the MATLAB Simulink file
6. Change the COM# on the serial configuration, serial receive, and serial send blocks

Setup

Arduino Sketch (Lab_2_Arduino_Setup.ino):

- Read the value from the scale and write it to Simulink.
- Read the set point power percentage from Simulink to set the pump power.
- Be sure to download the Adafruit NAU7802 Library in the Arduino IDE.

MATLAB Simulink Simulation (Lab_2_Simulink_Setup.slx):

- Receive the scale value from Arduino for visualization and calibration.
- Send the power percentage for the outlet pump to Arduino to further set the pump power.

Getting System Response Data

Arduino Sketch (Lab_2_Arduino.ino):

- Read the value from the scale and write it to Simulink.
- Calculate and set the outlet pump power according to the fullness of the beaker.
- Receive the inlet pump power from Simulink to set the pump power.

MATLAB Simulink Simulation (Lab_2_Simulink.slx):

- Receive the scale value from Arduino for data logging.
- Send the power percentage for the inlet pump to Arduino to further set the pump power.

MATLAB Data Processing and Saving (Lab_2_MATLAB.m):

- Visualize and save system response data to a csv file for later modeling.

Procedure

1. Make the Arduino connection with the scale, pumps, and breadboard.
 - Be sure to have a TA check everything is wired correctly.
2. Connect the Arduino to your computer.
3. Open the "Lab_2_Arduino_Setup.ino" file.
4. Select the correct COM port in the Arduino IDE and upload the code to the Arduino.
5. Open the "Lab_2_Simulink_Setup.slx" file in Simulink and change the COM ports accordingly.
6. Run the file to check if the scale is working properly.
7. If the scale is running properly, put an empty beaker with a pump in it on the scale and record the measurement.

8. Fill the beaker to the 500 mL mark with water and the pump in it and record the measurement.
9. Open the "Lab_2_Arduino.ino" file in Arduino IDE.
10. Change the scale reading (empty and full) with your recorded scale readings in the code.
11. Set up all the equipment as shown in the diagram and fill both beakers to around 300 mL.
 - Ensure that the pump is fully submerged in water and at the bottom of the beakers.
 - Place one beaker on the scale with the outlet pump and the other beaker without the scale with the inlet pump. The inlet pump works as the input to the system.
12. Before proceeding, have one of the TAs check everything is set up properly.
13. Select the correct COM port.
14. Upload the Arduino code.
15. Open the "Lab_2_Simulink.slx" Simulink file and change the COM ports accordingly.
16. Start with 100% power of the inlet pump. Set 100 as the input value in the Simulink file.
17. Run the file and let it reach a steady state.
 - Steady state means the beaker on the scale has no significant change in level.
18. While the simulation is running, give a step change to the input by changing the power of the inlet pump to 70%.
19. Allow everything to reach a steady state and then stop the simulation.
20. Run the MATLAB file "Lab_2_MATLAB.m" to visualize and save the system dynamics with the step input.

Submission

Once you have completed the data acquisition, you will be able to generate a model of the system. Use the dynamic data to estimate the transfer function using the System Identification Toolbox to determine if it is a first-order, second-order, or has a time delay.

You may need to filter your data for outliers and delete points where data was not collected.

A MATLAB file, “Lab_2_MATLAB_Data.m” is available to help get your into MATLAB for more processing.

The submission should include two files:

1. The data file as a csv
2. A pdf including:
 - A description of the experiment
 - Explanation of any filtering done
 - Model fitting with code and explanation
 - Selection of best model
 - Plot of the input and output data compared to the model